Compras-Backend

No puedes darte de baja a ti mismo y la contraseña no se cambia al editar

Documento explicativo · 9 secciones + 3 apéndices · generado el 11 de octubre de 2026

> **Este documento NO modificó nada del proyecto**
> Todo lo que sigue es resultado de **leer** el código de los dos repositorios. No se creó, no se borró y no se editó ningún archivo. El código que aparece en las secciones 5 a 8 es la **propuesta**: el lector lo aplica a mano, y por eso viene completo, con el *oldString*/*newString* exacto y la línea donde va.

Que se explica	Donde
Mejora 1 — no puedes darte de baja a ti mismo	Backend `UserImpl` + Angular `usuarios.ts/.html/.css`
Mejora 2 — el input de contraseña se deshabilita al editar	Angular `usuarios.html/.ts` (el backend ya era immune)
Endurecimiento opcional — rechazar el password en el PUT	`UpdateUserRequest` + `UserImpl`
Como se verifica	Un test de servicio nuevo + la suite de 294 tests

1. Las dos mejoras, en una frase cada una

Mejora 1. En la tabla de `/usuarios`, la fila que es **tu propio usuario** no se puede dar de baja. El boton sigue ahi, pero se ve bloqueado y al pulsarlo sale un Snackbar que explica por que: *«No puedes darte de baja a ti mismo»*.

Mejora 2. En el formulario de `/usuarios`, cuando estas en modo edicion el campo de contrasena aparece **deshabilitado**: se ve, pero no se puede escribir. Y en el backend, mandar un `password` en el PUT devuelve **400** en vez de ignorarse en silencio.

> **Por que el backend va primero en las dos**
> El proyecto ya tiene escrito este criterio (AGENTS.md, seccion *Validaciones*): **el frontend es la UX, el backend es la frontera de confianza**. Un boton deshabilitado no impide que alguien llame al `DELETE /api/local/users/1` desde Postman, curl o un script. El frontend es para que el usuario entienda; el backend es para que la regla sea verdad. Se hacen los dos, y por eso el trabajo se reparte entre los dos repositorios.

2. Que hay hoy (leido del codigo, no supuesto)

Antes de proponer nada hay que decir que existe y que no existe. Esto es lo que se encontro leyendo las clases.

2.1 Mejora 1: no hay NADA de proteccion

El endpoint existe y esta completo:

```
// UserController.java:58-63
@PreAuthorize("hasAuthority('DESACTIVAR_USUARIOS')")
@DeleteMapping("/{id}")
public void deactivateUser(@PathVariable Long id){
    service.deactivateUser(id);
}
```

Y el servicio recibe **solo el id**. No hay forma de que sepa quien esta llamando, porque nadie se lo dice:

```
// UserImpl.java:143-162  (version actual, completa)
@Override
public void deactivateUser(Long id) {
    UserEntity user = repository.findById(id).orElseThrow(() ->
        new UserException("Usuario no encontrado"));

    if(user.isActive()){
        user.setDeactivatedAt(LocalDateTime.now());
    }

    user.setActivatedAt(null);
    user.setActive(false);
    repository.save(user);
}
```

Ese metodo no importa `SecurityContextHolder`, no recibe `Authentication` y no busca al usuario logueado. **La auto-darbaja es hoy una operacion perfectamente legal.** Un ADMIN puede darse de baja a si mismo y la siguiente peticion ya vuelve con 401, porque el token se invalida al ver `active=false`.

2.2 Mejora 2: el backend YA es immune (y nadie lo escribio a proposito)

Esta es la sorpresa. `UpdateUserRequest` **no tiene campo password**:

```
// model/dto/User/UpdateUserRequest.java  (completo)
public class UpdateUserRequest {
    private String name;
    private String email;
    private Long roleId;
    private boolean active;

    // ... getters y setters
}
```

Y `UserImpl.updateUser()` nunca llama a `setPassword`. La consecuencia es que si alguien manda un `"password"` en el body del PUT, Jackson **lo descarta en silencio** (Spring Boot tiene `FAIL_ON_UNKNOWN_PROPERTIES=false` por defecto) y la operacion responde 200 como si nada.

El frontend tampoco lo manda. `usuarios.ts:196-203`:

```
if(this.form.id){
  this.userservice.updateUser(this.form.id,
    {
      name: this.form.name,
      email: this.form.email,
      roleId: this.form.roleId
    }
  ).subscribe({ ... });
}
```

Donde	Estado	Veredicto
`UpdateUserRequest` sin campo password	Ya correcto	No se toca
`UserImpl.updateUser()` sin setPassword	Ya correcto	No se toca
El frontend omite password al editar	Ya correcto	No se toca
El **input** de contrasena visible y escribible al editar	**El bug**	Se deshabilita
Un password en el PUT se ignora en silencio	Mentira silenciosa	Se rechaza con 400 (opcional)

El criterio que guió la mejora 2

Hay una diferencia entre *la regla se cumple* y *la regla se cumple y ademas te lo dice*. Hoy las dos mitades existen por separado: la regla se cumple (backend sin password) y la UI miente (input escribible). Arreglar la UI es lo que pediste. Anadir el 400 es el extra que hace que un cliente que se equivoque se entere en vez de creer que cambio algo.

3. De donde sale el usuario logueado (el dato que no existia)

Para bloquear la auto-darbaja hace falta responder una pregunta que el metodo actual no puede: **¿quien esta haciendo esta peticion?** Ese dato ya esta en el sistema, solo faltaba leerlo. Se lee en `config/JwtFilter.java`.

3.1 El principal ES un UserEntity, no un UserDetails

Este es el punto que decide como se escribe el codigo, asi que conviene entenderlo. `JwtFilter` no autentica con un `org.springframework.security.core.userdetails.User` sino con la **entidad completa**:

```
// config/JwtFilter.java:72-86
if(jwtUtil.isTokenValid(token)){
    if(email != null && SecurityContextHolder.getContext().getAuthentication() == null){

        UserEntity user = userRepository
            .findByEmailWithRoleAndPermissions(email).orElse(null);

        if(user != null){
            UsernamePasswordAuthenticationToken authToken =
                new UsernamePasswordAuthenticationToken(
                    user, null, authorityMapper.mapAuthorities(user));

            SecurityContextHolder.getContext().setAuthentication(authToken);
        }
    }
}
```

■ De ahi salen tres consecuencias practicas:

- **El id ya esta cargado.** Se puede comparar `actual.getId().equals(id)` sin consultar la base: es un entero que ya venia en memoria. Cero queries extra.
- **Se relee en cada peticion.** El filtro consulta la base siempre, asi que si un usuario se da de baja, su proximo request ya no autentica. Eso significa que **el backend ya tiene la garantia de que un dado de baja pierde el acceso en su siguiente peticion**: el unico agujero era poder darse de baja uno mismo.
- **El JWT no lleva el id.** El subject es el correo (`JwtUtil.generateToken`), no el id. Por eso no se puede comparar parseando el token: hay que leer el principal.

3.2 Ya existe un metodo identico en SaleImpl (el precedente que se copia)

`SaleImpl.java:260-305` ya resuelve «quien esta haciendo esta peticion» para poder poner `sale.setUser(...)`. Es exactamente el problema que se repite aqui:

```
// service/impl/SaleImpl.java:282-305
private UserEntity currentUserOrNull(){
    Authentication auth = SecurityContextHolder.getContext().getAuthentication();

    if(auth == null || !auth.isAuthenticated()){
        return null;
    }

    Object principal = auth.getPrincipal();

    // Spring pone "anonymousUser" (un String) cuando no hay token.
    if(principal instanceof UserEntity user){
        return user;
    }

    // Respaldo si algun dia el filtro cambia a un UserDetails.
    if(principal instanceof UserDetails details){
        return userRepository.findByEmail(details.getUsername()).orElse(null);
    }

    return null;
}
```

Decision: se COPIA el metodo, no se extrae a un util compartido

`InputValidator` (en `util/`) es estatico y no depende de nada, por eso vive ahi. Este metodo necesita un `UserRepository`, asi que ya no es una funcion pura. Sacarlo a `util/` obligaria a cambiar tambien `SaleImpl`, y con el una clase que hoy funciona y que tiene test. **Copiar 20 lineas es mas barato que refactorizar dos clases para no repetir 20 lineas.** Si manana hay un tercer sitio, ahi si se extrae.

3.3 Por que NO se hace en el controller

La otra opcion era pasar el usuario desde el controller, inyectando `Authentication` como parametro (que es lo que hace `AuthController.changePassword` en la linea 58). **No se eligio, y hay una razon concreta que solo se ve al correr los tests:**

```
// UserControllerTest.java:120-127  (test que YA existe)
@Test
@WithMockUser(authorities = "DESACTIVAR_USUARIOS")
void desactivarUsuario_devuelve200() throws Exception {
    mockMvc.perform(delete("/api/local/users/1"))
            .andExpect(status().isOk());

    verify(userService).deactivateUser(1L);
}
```

`@WithMockUser` mete en el principal un `org.springframework.security.core.userdetails.User`, **no** un `UserEntity`. Si el controller hiciera `(UserEntity) auth.getPrincipal()`, estos tests reventarian con `ClassCastException`: habria que reescribirlos todos.

Ademas hay una razon de arquitectura: **si el controller sabe quien es el usuario y se lo pasa al servicio, el servicio es una funcion pura que depende de que su llamador sea honesto.** Con el chequeo dentro del servicio, la regla se cumple aunque mañana alguien la llame desde otro endpoint, desde un job programado o desde un test.

Ubicacion del chequeo	Tests existentes	Regla real
Controller (inyectar `Authentication`)	■ **Se rompen** (ClassCastException)	Depende del llamador

Ubicacion del chequeo	Tests existentes	Regla real
Servicio (leer el SecurityContext) → **elegido**	■ Siguen en verde (`UserService` esta mockeado)	Se cumple siempre

4. Mapa de archivos que cambian

4.1 Backend (Compras_Backend)

Archivo	Cambio	Obligatorio
`service/impl/UserImpl.java`	Guarda de auto-darbaja en `deactivateUser` + metodo privado `currentUserOrNull()` + guarda de password en `updateUser`	**Si**
`model/dto/User/UpdateUserRequest.java`	Agregar campo `password` (solo para rechazarlo)	No (opcional)
`controller/UserController.java`	Ninguno. Se documenta **por que**	—
`test/.../service/impl/UserImplAutoBajaTest.java`	Test nuevo (5 casos)	**Si**

4.2 Frontend (Compras_Frontend-Angular)

Archivo	Cambio	Obligatorio
`features/users/usuarios.ts`	Getter `esYo()`, guarda en `deactivateUser()`, error visible con Snackbar, salida temprana en `validarPassword()`	**Si**
`features/users/usuarios.html`	**[disabled]** en el input de password, clase `.blocked` en el boton, etiqueta «`tú`»	**Si**
`features/users/usuarios.css`	Tres reglas nuevas (input deshabilitado, boton bloqueado, etiqueta)	**Si**
`features/users/usuarios.spec.ts`	Dos tests nuevos de `esYo()`	No (opcional)

Lo que NO se toca, y por que

`UserController`, `UserService` (la interfaz), `UserEntity`, `UserMapper`, `UserRepository` y `exceptions/UserException.java`. **No hace falta ninguna migracion de Flyway** ni ningun permiso nuevo: `UserException(String, HttpStatus)` ya existe y `GlobalExceptionHandler` ya lo traduce a `USER_ERROR`. Tampoco se toca `deactivated-users.ts`: esa pantalla solo da de *alta*, y darse de alta a si mismo no es un problema.

5. Backend - UserImpl.java (los 3 cambios)

5.1 Los imports que se agregan

Van despues de `import org.springframework.stereotype.Service;`:

```
import org.springframework.security.core.Authentication;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.userdetails.UserDetails;
```

> **Por que NO se importa HttpStatus**
>
> Este archivo ya usa la forma larga dos veces: `new UserException(…, org.springframework.http.HttpStatus.CONFLICT)` (lineas 77 y 128). El cambio sigue esa convencion para no tener dos estilos en el mismo archivo. Si prefieres limpiarlo, se agrega el import y se acortan las tres: es cosmetico.

5.2 La guarda de auto-darbaja

Se inserta en `UserImpl.deactivateUser()`, replacing el `public void deactivateUser(Long id) {` actual:

```
    //Desactivar usuario
    @Override
    public void deactivateUser(Long id) {
        // V8: nadie se da de baja a si mismo.
        //
        // El bloqueo va AQUI y no solo en el boton del frontend por una razon
        // concreta: el frontend es la UX, el backend es la frontera de confianza.
        // Un boton deshabilitado no impide que alguien llame al DELETE desde
        // Postman o desde un script, y el problema real de la auto-darbaja no
        // es el clic equivocado: es que la operacion quede LEGAL.
        //
        // Va ANTES del findById a proposito: la pregunta es sobre QUIEN llama,
        // no sobre el usuario objetivo. Ademas ahorra un SELECT.
        UserEntity actual = currentUserOrNull();

        if(actual != null && actual.getId().equals(id)){
            throw new UserException(
                "No puedes darte de baja a ti mismo. Pide que otro administrador lo haga.",
                org.springframework.http.HttpStatus.CONFLICT);
        }

        UserEntity user = repository.findById(id).orElseThrow(() ->
            new UserException("Usuario no encontrado"));

        // ... el resto del metodo SIN CAMBIOS
    }
```

Por que 409 y no 403 ni 400

Codigo	Que significa	Encaja aqui?
409 CONFLICT → elegido	La peticion es valida en forma pero choca con el estado del mundo	Si: el estado es «este usuario esta dado de baja»
403 FORBIDDEN	No tienes permiso para esto	Mentira: `DESACTIVAR_USUARIOS` si lo tienes
400 BAD REQUEST	El request esta mal formado	El request esta perfecto; lo que choca es la regla de negocio

Por que se compara el ID y no el correo

El correo puede cambiar: un usuario se puede editar su correo y en ese momento compararlo por correo daria un falso negativo. El `id` no cambia nunca, y ya esta en memoria en el principal. Ademas el backend normaliza el correo a minusculas al guardar, asi que una comparacion por cadena seria otra fuente de falsos negativos.

5.3 El metodo privado currentUserOrNull()

Va justo antes de `activateUser()` (despues de `deactivateUser()`), copiando el de `SaleImpl`:

```
    /**
     * Usuario autenticado que esta haciendo esta peticion, o {@code null}.
     *
     * <p>Se lee del {@code SecurityContext}, que es donde Spring Security deja
     * al usuario del JWT ya validado. No se pide por parametro: si el cliente
     * pudiera decir quien es, el bloqueo de la seccion 5.2 no valdría nada.
     *
     * <p><b>El principal es un {@code UserEntity}, no un {@code UserDetails}.</b>
     * Eso lo define {@code JwtFilter}, que autentica con la entidad completa
     * (rol y permisos incluidos). Por eso se lee el principal <b>como entidad</b>
     * y no se busca al usuario en la base: ya esta cargado, y volver a
     * consultarlo seria un SELECT extra por peticion.
     *
     * <p>Es el mismo metodo que ya existe en {@code SaleImpl} (linea 282). Se
     * copio en vez de extraerse a un util compartido porque este necesita un
     * {@code UserRepository}: al sacarlo habria que tocar tambien {@code SaleImpl}.
     *
     * <p>Si no hay usuario autenticado devuelve {@code null} en vez de fallar,
     * igual que en {@code SaleImpl}. Es lo que hace que los tests unitarios
     * (que no levantan el filtro de seguridad) sigan funcionando sin montar
     * nada: sin sesion no hay auto-darbaja que bloquear.
     */
    private UserEntity currentUserOrNull(){
        Authentication auth = SecurityContextHolder.getContext().getAuthentication();

        if(auth == null || !auth.isAuthenticated()){
            return null;
        }

        Object principal = auth.getPrincipal();

        // Spring pone "anonymousUser" (un String) cuando no hay token: no es un
        // usuario real y no debe buscarse.
        if(principal instanceof UserEntity user){
            return user;
        }

        // Respaldo por si algun dia el filtro cambia a un UserDetails (por
        // ejemplo al migrar a un AuthenticationProvider). Con esta rama el
        // servicio sigue funcionando sin tocarlo.
        if(principal instanceof UserDetails details){
            return repository.findByEmail(details.getUsername()).orElse(null);
        }

        return null;
    }
```

El null es a proposito, no una■■mildad

■ Si `currentUserOrNull()` devolviera `null` al no haber sesion y el metodo `deactivateUser` lo tratara como error, los 6 tests de `UserControllerTest` y los de `UserImplCamposObligatoriosTest` tendrian que montar un SecurityContext. El coste de devolver `null` es que, sin sesion, la operacion pase. **Eso no es un agujero real**: todo endpoint exige `.anyRequest().authenticated()`, asi que sin sesion la peticion muere en el filtro antes de llegar aqui.

6. Backend - el rechazo de la contrasena (endurecimiento opcional)

6.1 El campo trampa en UpdateUserRequest

Se agrega el campo **unicamente para poder rechazarlo**, junto a los existentes:

```
public class UpdateUserRequest {
    private String name;
    private String email;
    private Long roleId;
    private boolean active;

    /**
     * Campo trampa: NO se usa para cambiar la contrasena.
     *
     * <p>Existe solo para RECHAZAR un "password" que llegue en el body, en vez
     * de ignorarlo en silencio. Sin este campo, Jackson descarta la propiedad
     * desconocida (FAIL_ON_UNKNOWN_PROPERTIES=false) y la peticion responde 200:
     * el cliente cree que cambio la contrasena y no cambio nada.
     *
     * <p>La contrasena se cambia por su propio camino, que exige la actual:
     * POST /api/local/auth/change-password (AuthController:57).
     */
    private String password;

    // ... getters y setters, incluido:
    public String getPassword(){ return password; }
    public void setPassword(String password){ this.password = password; }
}
```

6.2 La guarda en updateUser()

Va al principio de `updateUser()`, antes del `findById`:

```
    //Actualizar usuario
    @Override
    public UserDto updateUser(Long id, UpdateUserRequest request) {
        // V8: editar un usuario NO cambia la contrasena. Ver seccion 6.
        if(request.getPassword() != null && !request.getPassword().isBlank()){
            throw new UserException(
                "La contrasena no se cambia al editar un usuario. "
                + "Usa 'Cambiar contrasena' desde el perfil.",
                org.springframework.http.HttpStatus.BAD_REQUEST);
        }

        UserEntity user = repository.findById(id).orElseThrow(() ->
            new UserException("Usuario no encontrado"));

        // ... el resto del metodo SIN CAMBIOS
    }
```

Por que el chequeo es !isBlank() y no != null

Lo que llega	Con `!= null`	Con `!isBlank()`
`null` (el caso normal)	pasa	pasa
`""`	■ **400 falso**	pasa
`" "`	■ **400 falso**	pasa

Lo que llega	Con `!= null`	Con `!isBlank()`
`"nueva123"`	400	400

El frontend actual manda el body sin la propiedad, asi que llega `null` y la guarda nunca se dispara. Los dos casos intermedios son los que importan: un cliente viejo o un formulario que serialice el campo vacio no deben recibir un error que no se puede explicar.

Se puede NO hacer esto

Los dos cambios de esta seccion son opcionales. La mejora 2 que pediste se resuelve **entera en el frontend**, porque el backend ya era immune. El 400 es solo para que un cliente mal informado se entere en vez de creer que funciono. Si prefieres el diff minimo, aplica solo la seccion 6 y recuerda que el campo trampa no es funcional: es una honestidad del contrato de la API.

7. Frontend - usuarios.ts

7.1 El getter esYo()

Va junto a los otros getters del componente, despues de `filteredUsers` (linea 55):

```
  /**
   * El usuario que tiene la sesion abierta.
   *
   * <p>Viene del localStorage (AuthService.getUser()), no del backend: es el
   * mismo dato que ya usa `auth.hasPermission(...)` en el template, asi que no
   * agrega una fuente de verdad nueva ni una peticion.
   */
  get currentUserId(): number | null {
    return this.auth.getUser()?.id ?? null;
  }

  /**
   * ¿La fila de la tabla es el usuario que esta logueado?
   *
   * <p>Se compara por ID y no por correo: el correo se puede editar y el id no
   * cambia nunca.
   *
   * <p>■ El `!= null` NO es opcional. Los dos lados son `number | undefined`
   * (el `id` de la interface `User` es opcional), asi que sin el guard una fila
   * sin id compararia `undefined === undefined` y daria TRUE: se bloquearia la
   * fila equivocada.
   */
  esYo(user: User): boolean {
    const id = this.auth.getUser()?.id;
    return id != null && user.id === id;
  }
```

7.2 La guarda en deactivateUser()

El metodo actual (lineas 269-282) cambia de firma: de `deactivateUser(id?: number)` a `deactivateUser(user: User)`.

> **Por que cambiar la firma en vez de dejar el id**
>
> El metodo necesita el **usuario completo** para responder «soy yo». Comparar ids contra `this.currentUserId` tambien funcionaria, pero obliga al template a pasar dos cosas. Pasando el objeto, el template queda en `(click)="deactivateUser(p)"` — igual que `editUser(p)`, que ya recibia el objeto. **Un solo parametro y una sola forma de llamar.**

```
  //Da de baja (borrado logico) un usuario. Como esta tabla solo lista activos,
  //el usuario se quita de la lista: aparecera en /usuarios-desactivados.
  //
  // V8: recibe el USER y no el id para poder compararlo con el de la sesion.
  deactivateUser(user: User){
    if(!user.id) return;

    // No te puedes dar de baja a ti mismo. El aviso va ANTES del confirm() a
    // proposito: pedirle confirmacion al usuario de algo que no va a pasar es
    // hacerle perder un clic.
    if(this.esYo(user)){
      this.snack.open(
        'No puedes darte de baja a ti mismo. Pide que otro administrador lo haga.',
        'Cerrar', { duration: 4000 });
      return;
    }

    if(confirm('\u00bfSeguro que deseas dar de baja este usuario?')){
      this.userservice.deActiveUser(user.id).subscribe({
        next: () => {
          this.users = this.users.filter(u => u.id !== user.id);
        },
        error: (err) => {
          // El backend responde 409 si el token corresponde al mismo usuario que
          // se intenta dar de baja. Antes esto solo iba a la consola y el
          // usuario veia que no paso nada.
          this.snack.open(
            err.error?.message || 'No se pudo dar de baja el usuario',
            'Cerrar', { duration: 4000 });
        }
      });
    }
  }
```

Se quita el `console.log('Usuario dado de baja')` y el `console.error`: el primero era ruido y el segundo es justo el bug que se arregla (un error que solo existe en la consola del desarrollador no es un error que el usuario pueda ver).

7.3 La salida temprana en validarPassword()

Reemplaza el metodo actual (lineas 120-148). El comportamiento observable es el mismo, pero la regla queda en una sola linea en vez de repartida en un `if(this.form.id)` anidado:

```
  /**
   * Valida la contrasena.
   *
   * <p>V8: al editar el input esta DESHABILITADO, asi que nunca llega texto
   * nuevo. La funcion no se borra porque el mismo componente la llama al CREAR,
   * y porque es la que deja el formulario coherente si alguien vuelve a
   * habilitar el input desde otro lado.
   *
   * <p>Solo es obligatoria al CREAR. Exigirla al editar dejaria imposible
   * guardar cualquier edicion, porque al editar la contrasena no se manda.
   */
  validarPassword(){
    if(this.form.id){
      delete this.errores['password'];
      return;
    }

    const pw = this.form.password ?? '';

    if(!pw){
      this.errores['password'] = 'La contrasena es obligatoria.';
      return;
    }

    const largo = pw.trim().length;
    if(largo < this.MIN_PASSWORD_LENGTH){
      this.errores['password'] =
        `Minimo ${this.MIN_PASSWORD_LENGTH} caracteres (llevas ${largo}).`;
    } else if(largo > this.MAX_PASSWORD_LENGTH){
      this.errores['password'] =
        `Maximo ${this.MAX_PASSWORD_LENGTH} caracteres (llevas ${largo}).`;
    } else {
      delete this.errores['password'];
    }
  }
```

8. Frontend - usuarios.html y usuarios.css

8.1 El input de contrasena

Se reemplaza el bloque de las lineas 30-41. El unico cambio real es `[disabled]="{{form.id}}"`:

```
        <div class="field">
          <label>Contrasena</label>
          <!-- minlength documenta la regla en el propio campo; el bloqueo real
               lo hace save() en usuarios.ts (MIN_PASSWORD_LENGTH), que ademas
               avisa cuantos caracteres lleva.

               V8: [disabled] cuando hay id, o sea en modo edicion. Se VE el
               campo en vez de esconderlo: "no aparece" parece un forgot del
               formulario, mientras que "aparece pero no se puede escribir"
               comunica que es una regla del sistema, no un error de la pagina.
               Un input disabled tampoco dispara (input), asi que validarPassword()
               nunca ve texto en modo edicion. -->
          <input name="form.password" [(ngModel)]="form.password" type="password"
                 minlength="8" maxlength="10" autocomplete="new-password"
                 [disabled]="!!form.id"
                 (input)="validarPassword()"
                 [class.input-error]="errores['password']"/>
          <span class="error-msg" *ngIf="errores['password']">{{ errores['password'] }}</span>

          <small class="hint" *ngIf="!form.id">Minimo 8 caracteres</small>
          <small class="hint" *ngIf="form.id">
            La contrasena no se cambia aqui. Usa "Cambiar contrasena" en tu perfil.
          </small>
        </div>
```

Por que se ve en vez de ocultarse con *ngIf

Con `*ngIf="!form.id"` el campo desapareceria y el formulario encogeria de golpe al pulsar Editar: el layout se mueve y el resto de campos saltan de lugar. Ademas el usuario no tendria forma de saber *por que* no hay contrasena. El texto de abajo lo explica.

8.2 El boton Dar de baja

Se reemplaza el boton de las lineas 125-127:

```
                    <!-- V8: el boton NO lleva [disabled]. Si llevara, el
                         click nunca llegaria a deactivateUser() y el Snackbar
                         con el motivo nunca se veria. Lo que se hace es
                         marcarlo visualmente con .blocked y dejar que el clic
                         cuente la historia: el usuario pulsa, se le explica.
                         (La alternativa [disabled] + solo title tambien es
                         valida; es la que muestra el aviso en un tooltip.) -->

                    <button *hasPermission="'DESACTIVAR_USUARIOS'" type="button"
                            (click)="deactivateUser(p)"
                            [class.blocked]="esYo(p)"
                            [title]="esYo(p) ? 'Eres tu: no puedes darte de baja' : 'Dar de baja a este usu
                            class="delete">
                      Dar de baja
                    </button>
```

Por que [class.blocked] y no [disabled]

El elemento ya lleva `*hasPermission`, que es una **directiva estructural**: crea y destruye la vista. Angular solo permite **una** directiva `*` por elemento, asi que un `*ngIf="!esYo(p)"` al lado directamente no compila. `[class.x]` y `[disabled]` si son propiedades y se pueden combinar. Entre las dos, `[class.blocked]` es la que deja que el mensaje aparezca.

8.3 La etiqueta tu en la fila propia

En la celda del nombre (linea 115):

```
                <td>
                  {{ p.name }}
                  <!-- V8: marca visible de que esa fila eres tu. Es lo que
                       hace evidente el bloqueo antes de que el usuario intente
                       pulsarlo, y de paso no hace falta que el boton este ahi
                       deshabilitado para que se entienda. -->
                  <span class="yo-tag" *ngIf="esYo(p)">tu</span>
                </td>
```

8.4 Las tres reglas de CSS

Se anexan al final de `usuarios.css`. El orden importa y esta anotado:

```
/* ================================================================
   V8 - Estado bloqueado (tu propio usuario)
   El orden de estas reglas RESPETA el orden del archivo: .delete.blocked
   va DESPUES de .delete:hover porque tienen la misma especificidad
   (0,2,0) y en CSS gana la que se escribio al final.
   ================================================================ */

/* El boton "Dar de baja" de tu propia fila: se ve bloqueado pero NO esta
   disabled, asi que el clic llega y el Snackbar puede explicar el motivo.
   Sin esto el boton se veria igual de disponible que los demas y el usuario
   solo descubriria el bloqueo al pulsarlo. */
.delete.blocked {
  background: linear-gradient(135deg, #334155, #1e293b);
  color: #94a3b8;
  cursor: not-allowed;
}

.delete.blocked:hover {
  background: linear-gradient(135deg, #334155, #1e293b);
  transform: none;
}

/* Input de contrasena deshabilitado al editar.
   ■ Esta regla es OBLIGATORIA: .form-card input ya pone fondo #020617 y
   borde #334155, y el navegador NO cambia el estilo de un input disabled
   al que tiene reglas propias. Sin esto el campo deshabilitado se veria
   exactamente igual que uno habilitado y el usuario no notaria nada. */
.form-card input:disabled {
  background: #1e293b;
  border-color: #334155;
  color: #64748b;
  cursor: not-allowed;
  opacity: 0.75;
}

/* Etiqueta "tu" en la fila propia */
.yo-tag {
  display: inline-block;
  margin-left: 6px;
  padding: 1px 7px;
  border-radius: 999px;
  background: #1e40af;
  color: #dbeafe;
  font-size: 10px;
  font-weight: 700;
  letter-spacing: .4px;
  vertical-align: middle;
}
```

El detalle que hace que esto se vea bien

`.form-card input` (linea 52 de `usuarios.css`) pone `background: #020617` y `border: 2px solid #334155`. El navegador no reemplaza el estilo de un control deshabilitado al que se le aplican reglas propias: solo aporta el cursor por defecto. O sea que sin la regla `input:disabled` el campo se veria **identico** a uno editable y el cambio seria invisible. Es el mismo motivo por el que existe el boton *Guardar usuario* con sombra en el modulo de compras.

9. Verificacion

9.1 El test de servicio nuevo (backend)

Archivo:
`src/test/java/com/erikjarquin/compras/service/impl/UserImplAutoBajaTest.java`

```
package com.erikjarquin.compras.service.impl;

import static org.assertj.core.api.Assertions.assertThat;
import static org.assertj.core.api.Assertions.catchThrowableOfType;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.never;
import static org.mockito.Mockito.verify;
import static org.mockito.Mockito.when;

import java.util.List;
import java.util.Optional;

import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Nested;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter.MockitoExtension;
import org.mockito.junit.jupiter.MockitoSettings;
import org.mockito.quality.Strictness;
import org.springframework.http.HttpStatus;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.context.SecurityContextHolder;

import com.erikjarquin.compras.exceptions.UserException;
import com.erikjarquin.compras.model.dto.User.UpdateUserRequest;
import com.erikjarquin.compras.model.entity.UserEntity;
import com.erikjarquin.compras.repository.RoleRepository;
import com.erikjarquin.compras.repository.UserRepository;
```

Parte 2 de 4 - el andamiaje

```
/**
 * V8 - No te puedes dar de baja a ti mismo + la contrasena no se cambia al editar.
 *
 * <p>El SecurityContext se monta a mano con el mismo constructor que usa
 * JwtFilter: principal = UserEntity. Asi se prueba el camino real, no una
 * simulacion.
 */
@ExtendWith(MockitoExtension.class)
@MockitoSettings(strictness = Strictness.LENIENT)
@DisplayName("Usuarios - auto-darbaja y contrasena en edicion (V8)")
class UserImplAutoBajaTest {

    @Mock private UserRepository repository;
    @Mock private RoleRepository roleRepository;
    @Mock private org.springframework.security.crypto.password.PasswordEncoder passwordEncoder;

    @InjectMocks private UserImpl service;

    @AfterEach
    void limpiarContexto(){
        SecurityContextHolder.clearContext();
    }

    /**loguea a un usuario con el MISMO constructor que usa JwtFilter. */
    private void autenticarComo(UserEntity u){
        SecurityContextHolder.getContext().setAuthentication(
            new UsernamePasswordAuthenticationToken(u, null, List.of()));
    }

    private UserEntity usuario(Long id, String email){
        UserEntity u = new UserEntity();
        u.setId(id);
        u.setName("Erik");
        u.setEmail(email);
        u.setActive(true);
        return u;
    }
```

Parte 2 de 3 - la auto-darbaja

```
    @Nested
    @DisplayName("No puedes darte de baja a ti mismo")
    class AutoBaja {

        @Test
        @DisplayName("409 y NO escribe nada")
        void desactivarPropio_lanza409YNoGuarda(){
            UserEntity yo = usuario(7L, "yo@correo.com");
            autenticarComo(yo);
            when(repository.findById(7L)).thenReturn(Optional.of(yo));

            UserException e = catchThrowableOfType(
                () -> service.deactivateUser(7L), UserException.class);

            assertThat(e.getStatus()).isEqualTo(HttpStatus.CONFLICT);
            assertThat(e.getMessage()).contains("ti mismo");

            // Lo importante: no solo lanzar, sino NO HABER ESCRITO NADA.
            verify(repository, never()).save(any());
            verify(repository, never()).findById(any());
        }

        @Test
        @DisplayName("Dar de baja a OTRO si funciona")
        void desactivarOtro_siGuarda(){
            UserEntity yo = usuario(1L, "yo@correo.com");
            UserEntity otro = usuario(7L, "otro@correo.com");
            autenticarComo(yo);
            when(repository.findById(7L)).thenReturn(Optional.of(otro));

            service.deactivateUser(7L);

            assertThat(otro.isActive()).isFalse();
            assertThat(otro.getDeactivatedAt()).isNotNull();
            verify(repository).save(otro);
        }
    }

    @Nested
    @DisplayName("Por que el chequeo tolera no haber sesion")
    class SinSesion {

        @Test
        @DisplayName("Sin SecurityContext no bloquea (tests, llamadas internas)")
        void sinSecurityContext_noBloquea(){
            UserEntity otro = usuario(7L, "otro@correo.com");
            when(repository.findById(7L)).thenReturn(Optional.of(otro));

            service.deactivateUser(7L);

            assertThat(otro.isActive()).isFalse();
        }
    }
```

Parte 3 de 3 - la contrasena en edicion

```
    @Nested
    @DisplayName("La contrasena NO se cambia al editar")
    class PasswordEnEdicion {

        private UpdateUserRequest editConPassword(String pw){
            UpdateUserRequest r = new UpdateUserRequest();
            r.setName("Erik");
            r.setEmail("erik@correo.com");
            r.setRoleId(1L);
            r.setPassword(pw);
            return r;
        }

        @Test
        @DisplayName("Si mandan password -> 400 y no guarda")
        void editarConPassword_lanza400YNoGuarda(){
            UpdateUserRequest r = editConPassword("nuevaClave123");

            UserException e = catchThrowableOfType(
                () -> service.updateUser(7L, r), UserException.class);

            assertThat(e.getStatus()).isEqualTo(HttpStatus.BAD_REQUEST);
            verify(repository, never()).save(any());
        }

        @Test
        @DisplayName("Una contrasena vacia NO es error (clientes viejos)")
        void editarConPasswordVacia_siEdita(){
            UpdateUserRequest r = editConPassword("   ");
            UserEntity u = usuario(7L, "erik@correo.com");
            when(repository.findById(7L)).thenReturn(Optional.of(u));
            when(repository.findByEmail("erik@correo.com")).thenReturn(Optional.empty());
            when(roleRepository.findById(1L)).thenReturn(Optional.empty());

            // El unico assert que importa: no se guardo con password.
            service.updateUser(7L, r);

            verify(repository).save(u);
            assertThat(u.getPassword()).isNull();
        }
    }
}
```

El assert que de verdad protege el bug

■ `verify(repository, never()).save(any())`. Comprobar solo que se lanza la excepcion es insuficiente: si el metodo guardara *antes* de validar, el test pasaria y el dano estaria hecho. Es el mismo assert que se uso en `PurchaseImpl.cancel()` (ver la entrada del 2026-09-30 (2) de AGENTS.md).

9.2 La suite completa

```
.\mvnw.cmd test -Dtest='!ComprasApplicationTests'
```

Que deberia pasar	Por que
Los 294 tests siguen en verde	Los 6 de `UserControllerTest` no cambian: el servicio esta mockeado y nadie lo invoco
Los 15 de `UserImplCamposObligatoriosTest` siguen	Usan `@MockitoSettings(LENIENT)` y no montan SecurityContext

Que deberia pasar	Por que
Los 5 nuevos de `UserImplAutoBajaTest` pasan	Cubren 409 + no-escritura + el camino de 'dar de baja a otro'
`ng build` sin errores	El unico riesgo es el * doble en el boton, que esta evitado con `[class.blocked]`

9.3 Prueba manual (2 minutos)

Paso	Que se espera
Entrar como admin y abrir `/usuarios`	Tu fila tiene la etiqueta azul **tu**
Pulsar **Dar de baja** en TU fila	Sale el Snackbar *«No puedes darte de baja a ti mismo»* y **no** sale el `confirm()`
Pulsar **Dar de baja** en otra fila	Sale el `confirm()` normal
Pulsar **Editar** en cualquier usuario	El campo Contrasena esta visible, atenuado y no acepta escritura; el texto de abajo dice que se cambia desde el perfil
Guardar la edicion	Se guardan nombre, correo y rol; la contrasena no se toca
`curl -X DELETE /api/local/users/` con tu token	■ **409** con `USER_ERROR`. Esta es la prueba que demuestra que la regla es del backend

La prueba del curl no es opcional

Si solo se prueba la interfaz, no se sabe si el bloqueo esta en el boton o en el servidor. Con el `curl` se separa una cosa de la otra: si el `DELETE` responde 409, la regla es del backend. Si responde 200, lo unico que funciona es el boton y un script se la salta.

10. Apendice A - Flujo completo de la auto-darbaja

#	Quien	Que hace	Resultado
1	Admin	Abre `/usuarios`	`GET /api/local/users` → 200 con todos
2	JwtFilter	Valida el token	`UserEntity` completo en el principal
3	Template	Por cada fila llama `esYo(p)`	Tu fila recibe `.blocked` y la etiqueta **tu**
4	Admin	Pulsa **Dar de baja** en su fila	No hay `[disabled]`: el clic llega
5	`deactivateUser()`	`esYo(user)` → true	Snackbar + `return` (no se llama a la API)
6	*Escenario B: curl directo*		Se salta los pasos 3-5
7	`UserController.java:59`	`@PreAuthorize` con `DESACTIVAR_USUARIOS`	Pasa: el admin **sí** tiene el permiso
8	`UserController.java:61`	`deactivateUser(Long id)`	El controller no sabe quien es. Pasa el id tal cual
9	`UserImpl`	`currentUserOrNull()` → id 1 → `id`	■ **AQUI ESTA LA REGLA**
10	`UserImpl`	`UserException(409)`	409 + `USER_ERROR`
11	*Escenario C: otro admin te da de baja*	El principal es id 2, el objetivo es id 1	El chequeo pasa: la operacion es legitima

Escenario A = el boton. **Escenario B** = el mismo resultado por una via que no pasa por el boton. **Escenario C** = el que demuestra que la regla es *no tu mismo* y no *nadie*.

11. Apendice B - Deuda observada (NO se arregla aqui)

Leyendo las clases aparecieron tres cosas que no son parte del encargo. Se documentan para que no se pierdan y **no se tocan**, porque tocar un archivo es tocar su comportamiento y eso merece su propio encargo.

Hallazgo	Archivo	Por que NO se toca
`MAX_PASSWORD_LENGTH = 16` pero el input tiene `maxlength="10"`	`usuarios.ts:79` / `usuarios.html`	El codigo nunca avisara de una contrasena de 12 caracteres: el navegador la corta en 10 antes. Ademas el `minlength` de 8 si coincide con `MIN_PASSWORD_LENGTH`
`private boolean active` en `UpdateUserRequest`	`UpdateUserRequest.java:8`	`updateUser()` nunca lo lee: la baja tiene su propio endpoint. Es un campo muerto, pero borrarlo no cambia nada porque Jackson ya ignora lo que no esta mapeado
`resetForm()` deja `roleId: 0`	`usuarios.ts:238`	El `select` se queda sin opcion seleccionada hasta que `loadRoles()` solo lo rellena si `roleId === 0`. El arreglo es una linea, pero es un bug de *otra* pantalla de la misma historia

12. Apendice C - Checklist de aplicacion

#	Proyecto	Archivo	Cambio	Hecho
1	Backend	`UserImpl.java`	3 imports	■

#	Proyecto	Archivo	Cambio	Hecho
2	Backend	`UserImpl.java`	Guarda en `deactivateUser`	■
3	Backend	`UserImpl.java`	`currentUserOrNull()`	■
4	Backend	`.\mvnw.cmd compile`	Compila	■
5	Backend	`UserImplAutoBajaTest.java`	Test nuevo (5 casos)	■
6	Backend	`.\mvnw.cmd test -Dtest='!ComprasApplicationTests'`	294 + 5 en verde	■
7	Frontend	`usuarios.ts`	`esYo()` + guarda + Snackbar	■
8	Frontend	`usuarios.ts`	`validarPassword()` con salida temprana	■
9	Frontend	`usuarios.html`	`[disabled]` en password	■
10	Frontend	`usuarios.html`	`.blocked` + etiqueta **tu**	■
11	Frontend	`usuarios.css`	3 reglas nuevas	■
12	Frontend	`ng build`	Compila	■
13	Manual	Prueba del `curl`	Responde 409	■
14	*Opcional*	`UpdateUserRequest`	Campo trampa + guarda 400	■

Ultima cosa: donde queda el archivo

Este PDF vive en `Compras_Backend/docs/`, junto a los otros nueve de la serie. El generador (`generar_pdf10.py`, con reportlab) esta **fuera del repo**, en `%TEMP%\opencode\`, por la misma razon que los otros: el repo no necesita guardar como se escriben sus PDFs, solo los PDFs.